# 第02堂｜AI Agent 邏輯建構與自動化引擎配置

## 學習目標

完成本堂課後，學員將能夠：

- 理解 AI Agent 流程自動化工具的核心架構與底層邏輯
- 掌握「條件判斷」與「迴圈」這兩個自動化流程的基本控制元素
- 為自動化流程加入錯誤偵測與容錯補償機制
- 使用 GitHub Copilot 撰寫一個能與 LLM 對話的 Python 程式
- 讓 Agent 具備「工具調用」能力，理解 Agent 與普通對話的本質差異

---

## 核心概念說明

### 1. 自動化引擎的核心架構

一個自動化流程（不論是人工執行的 SOP，還是 AI Agent），都可以拆解為三個核心元素：

```
輸入（Input）
    │
    ▼
決策邏輯（Decision Logic）
  ├── 如果 A → 執行 X
  ├── 如果 B → 執行 Y
  └── 如果 C → 重複嘗試
    │
    ▼
輸出（Output）
```

對 AI Agent 而言，**決策邏輯**是由 LLM 動態生成的，而不是人工寫死的規則。

**AI Agent 自動化流程的完整運作週期：**

| 階段 | 說明 | 對應程式概念 |
|---|---|---|
| 接收任務 | 使用者或系統給出指令 | 函式的輸入參數 |
| 拆解步驟 | LLM 規劃執行計畫 | 演算法設計 |
| 執行行動 | 呼叫工具完成子任務 | 函式呼叫 |
| 確認結果 | 判斷子任務是否成功 | 條件判斷 |
| 錯誤處理 | 若失敗則嘗試替代方案 | 例外處理 |
| 回報完成 | 輸出最終結果 | 回傳值 |

---

### 2. 控制流：條件判斷（Condition）

**條件判斷**是自動化流程中最基本的決策機制：「如果滿足某個條件，就做 A；否則就做 B。」

**實際情境範例：**

你要設計一個自動回覆郵件的 Agent：

- **如果** 郵件主旨包含「請款」→ 轉交財務處理
- **如果** 郵件主旨包含「抱怨」→ 轉交客服主管
- **否則** → 自動生成標準回覆

**條件判斷的虛擬碼結構：**

```
如果 條件A 為真:
    執行動作A
否則如果 條件B 為真:
    執行動作B
否則:
    執行預設動作
```

**在 Agent 中的應用：**

```
如果 LLM 的決策 == "呼叫工具":
    執行指定工具
否則如果 LLM 的決策 == "任務完成":
    回傳最終結果
否則如果 LLM 的決策 == "需要更多資訊":
    詢問使用者補充資訊
```

---

## 3. 控制流：迴圈（Loop）

**迴圈**讓程式能夠**重複執行**某段邏輯，直到滿足特定條件為止。這在 Agent 中非常關鍵——Agent 必須持續「思考 → 行動 → 觀察結果 → 再思考」，直到任務完成。

**兩種常見的迴圈類型：**

| 類型 | 說明 | 應用場景 |
|---|---|---|
| **計數迴圈** | 執行固定次數後停止 | 批次處理 100 筆資料 |
| **條件迴圈** | 滿足條件才停止 | Agent 持續執行直到任務完成 |

**Agent 執行迴圈（條件迴圈）：**

```
重複執行 直到 任務完成:
    思考當前狀態
    決定下一個行動
    執行行動
```

```
    觀察結果
    更新對任務的理解
```

**批次處理迴圈（計數迴圈）：**

```
對 每一筆資料 在 資料清單 中:
    處理這筆資料
    儲存處理結果
```

> **注意：** 條件迴圈需要確保**終止條件**一定能被達到，否則程式會陷入「無限迴圈」，永遠無法停止。Agent 設計中通常會設定「最大嘗試次數」作為安全機制。

---

## 4. 錯誤處理與容錯補償機制

自動化系統在執行過程中，**錯誤幾乎是必然發生的**：網路連線中斷、API 回應逾時、資料格式不符.....等等。

好的自動化系統必須能夠：

1. **偵測錯誤**：知道哪裡出了問題
2. **分類錯誤**：是可以重試的暫時性錯誤，還是必須停止的嚴重錯誤？
3. **補償行動**：執行替代方案或通知人工介入

**常見的容錯策略：**

| 策略 | 說明 | 適用情境 |
|---|---|---|
| **重試（Retry）** | 自動重新嘗試，最多 N 次 | API 暫時逾時、網路不穩定 |
| **等待後重試（Exponential Backoff）** | 每次重試間隔越來越長 | API 呼叫頻率限制 |
| **替代方案（Fallback）** | 主要方案失敗時，使用備用方案 | 主要 LLM 故障時換用備用模型 |
| **人工介入通知** | 發送警告，請人工處理 | 無法自動修復的嚴重錯誤 |

---

## 5. 工具調用（Tool Use）：讓 Agent 能「做事」

單純的 LLM 只能「說話」，而 Agent 的強大之處在於能夠「做事」——這是透過**工具調用（Tool Use）**實現的。

**工具的概念：**

工具是一個有明確輸入與輸出的函式。Agent（LLM）在決策時，會選擇呼叫哪個工具、傳入什麼參數，然後根據工具的回傳結果決定下一步。

**常見工具類型：**

| 工具類型 | 功能 | 範例 |
|---|---|---|
| 搜尋工具 | 搜尋網路或資料庫 | Google Search API |

| 工具類型 | 功能 | 範例 |
|---|---|---|
| 檔案工具 | 讀取/寫入本機檔案 | 讀取 CSV、寫入 TXT |
| 計算工具 | 執行數學運算 | Python 計算機 |
| 通訊工具 | 發送郵件、訊息 | Outlook、Line API |
| 瀏覽器工具 | 操控網頁 | Playwright、Selenium |

**工具調用的完整流程：**

```
使用者任務 → LLM 分析任務
  │
  ├─ LLM 說「我需要使用搜尋工具，關鍵字為 XXX」
  │       │
  │       ▼
  │   程式執行搜尋工具(關鍵字="XXX")
  │       │
  │       ▼
  │   取得搜尋結果，傳回給 LLM
  │
  ├─ LLM 說「我需要寫入檔案，內容為 YYY」
  │       │
  │       ▼
  │   程式執行寫入檔案工具(內容="YYY")
  │
  └─ LLM 說「任務完成」→ 回傳最終答案
```

---

# 實作步驟（逐步教學）

## 實作一：用 GitHub Copilot 寫一個基本的 LLM 對話程式

**步驟 1：建立 Python 環境**

開啟 VS Code 終端機，依序輸入以下指令：

```
安裝 openai 套件
    pip install openai

確認安裝成功
    pip show openai
```

**步驟 2：設定 API Key 環境變數**

API Key 是存取 LLM 服務的身份憑證，**絕對不能直接寫在程式碼裡**（避免不小心上傳到 GitHub 等公開平台）。

1. 在 VS Code 終端機設定環境變數（Windows）：

```
set OPENAI_API_KEY=你的APIKey
```

2. 確認設定成功：

```
echo %OPENAI_API_KEY%
```

**步驟 3：向 GitHub Copilot 描述需求**

在 Copilot Chat 輸入：

```
請幫我用 Python 寫一個與 OpenAI GPT 對話的程式：
1. 從環境變數讀取 API Key，不要寫死在程式裡
2. 支援連續對話（保存對話歷史）
3. 使用者輸入 "exit" 就結束程式
4. 程式要有基本的錯誤處理（例如 API 連線失敗要顯示友善的錯誤訊息）
```

**步驟 4：檢視並理解生成的程式碼**

重點觀察：

- `messages` 變數如何保存對話歷史？
- `try / except` 是如何處理錯誤的？
- `os.environ` 是如何讀取環境變數的？

---

## 實作二：加入檔案工具，讓程式升級為 Agent

**步驟 1：向 Copilot 請求擴充功能**

```
請在剛才的程式基礎上，加入以下功能：
1. 定義兩個工具函式：read_file(path) 和 write_file(path, content)
2. 讓使用者可以在對話中要求 AI 讀取或寫入指定的文字檔案
3. AI 決定要用哪個工具時，由程式呼叫對應的函式，並把結果傳回給 AI
```

**步驟 2：測試 Agent 的工具調用**

用以下指令測試你的 Agent：

1. 「請幫我在桌面建立一個 test.txt，內容是今天的工作計畫。」
2. 「請幫我讀取桌面的 test.txt，告訴我裡面有什麼內容。」

**步驟 3：觀察與思考**

- Agent 如何決定要用哪個工具？
- 如果你給 Agent 的指令很模糊，它會怎麼處理？

- 如果工具執行失敗（例如檔案不存在），會發生什麼？

---

## 程式碼範例（虛擬碼）

### 含錯誤處理的 LLM 對話程式

```
# === 設定 ===
API_KEY = 從環境變數讀取("OPENAI_API_KEY")
最大重試次數 = 3
對話歷史 = [
    {"角色": "系統", "內容": "你是一個專業的工作助理，使用繁體中文回應"}
]

# === 含重試的 API 呼叫函式 ===
函式 呼叫LLM(對話歷史):
    對 嘗試次數 從 1 到 最大重試次數:
        嘗試:
            結果 = 呼叫 OpenAI API(對話歷史)
            回傳 結果
        如果發生 連線錯誤:
            顯示 f"連線失敗，第 {嘗試次數} 次重試..."
            等待 2 秒
        如果發生 其他錯誤:
            顯示 "發生未知錯誤，停止重試"
            拋出例外

    顯示 "已達最大重試次數，請檢查網路連線"
    結束程式

# === 主對話迴圈 ===
重複執行:
    使用者輸入 = 從終端機讀取()

    如果 使用者輸入 == "exit":
        結束程式

    將使用者輸入加入對話歷史
    AI 回應 = 呼叫LLM(對話歷史)
    將 AI 回應加入對話歷史
    顯示 AI 回應
```

### 帶工具調用的 Agent 虛擬碼

```
# === 定義工具 ===
函式 讀取檔案(檔案路徑):
    嘗試:
        內容 = 開啟並讀取(檔案路徑)
        回傳 內容
    如果發生錯誤:
```

```
        回傳 "錯誤：檔案不存在或無法讀取"

函式 寫入檔案(檔案路徑, 文字內容):
    嘗試:
        開啟檔案(檔案路徑) 並寫入 文字內容
        回傳 "成功寫入檔案"
    如果發生錯誤:
        回傳 "錯誤：無法寫入檔案"

工具清單 = {
    "讀取檔案": 讀取檔案,
    "寫入檔案": 寫入檔案
}

# === Agent 主迴圈 ===
函式 Agent執行(使用者指令):
    對話歷史 = [系統提示 + 工具說明 + 使用者指令]

    重複執行 最多 10 次:
        LLM回應 = 呼叫LLM(對話歷史)

        如果 LLM回應.類型 == "呼叫工具":
            工具名 = LLM回應.工具名稱
            參數   = LLM回應.工具參數
            工具結果 = 工具清單[工具名](參數)
            將工具結果加入對話歷史

        否則如果 LLM回應.類型 == "最終回答":
            回傳 LLM回應.內容

    回傳 "已達最大步驟數，任務未完成"
```

---

# 練習題 / 作業

## 課堂練習

**練習 1：流程設計**

設計一個「自動分類郵件」的 Agent 流程，用文字或流程圖描述：

- 輸入是什麼？（郵件主旨 + 內文）
- 需要哪些條件判斷？
- 可能發生哪些錯誤？要怎麼處理？
- 最終輸出是什麼？

**練習 2：判斷迴圈類型**

針對以下場景，判斷應使用「計數迴圈」還是「條件迴圈」，並說明理由：

| 場景 | 迴圈類型 | 理由 |
|---|---|---|

| 場景 | 迴圈類型 | 理由 |
|---|---|---|
| 處理一個資料夾內的 50 個 Excel 檔案 | | |
| 反覆呼叫 API 直到取得有效回應 | | |
| Agent 持續規劃執行直到任務完成 | | |
| 每天定時發送報表，執行 30 天 | | |

### 課後作業

1. **改良你的 Agent**：在課堂實作的程式基礎上，新增第三個工具「列出資料夾內的所有檔案」，讓 Agent 能夠知道某個資料夾有哪些檔案。
2. **錯誤情境測試**：故意製造錯誤情境（例如讀取一個不存在的檔案），觀察你的程式如何回應，並思考如何改進錯誤訊息讓它更清楚。

---

## 延伸閱讀

**Python 基礎：**

- Python 官方教學（中文） — https://docs.python.org/zh-tw/3/tutorial/
- W3Schools Python 教學 — https://www.w3schools.com/python/

**AI Agent 架構深入：**

- OpenAI Function Calling 文件 — https://platform.openai.com/docs/guides/function-calling
- ReAct：Agent 思考與行動框架論文（可搜尋「ReAct: Synergizing Reasoning and Acting in Language Models」）

**錯誤處理最佳實務：**

- Python 例外處理（Exception Handling）官方文件 — https://docs.python.org/3/tutorial/errors.html